

Kubernetes and Cloud Native Associate - KCNA

Cloud Native Architecture

Horizontal Pod Autoscaler

In this lesson, you'll learn about the Horizontal Pod Autoscaler (HPA) in Kubernetes and discover how it automatically adjusts the number of running pods based on the current resource usage. As its name implies, the HPA scales the pods horizontally—deploying more instances when demand increases and removing excess pods when demand decreases.



Key Concept

The HPA is one of Kubernetes' many controllers that works by adjusting the replica count of a Deployment. It scales up the replicas as demand grows and scales them down when the pressure eases.

How the HPA Works

The HPA monitors your application's resource consumption by leveraging metrics collected from a metrics server. This server continuously tracks resource usage across nodes and pods in the cluster, providing accurate data for the HPA to decide when to scale.

The HPA configuration typically involves:

- Defining a Deployment for the application.
- Setting resource limits and requests within the Deployment.
- Creating an HPA object that targets the Deployment.

Let's explore an example configuration.

Example Configuration

Below is an example YAML configuration for a Deployment that uses a container image with defined CPU resource limits:

```
# deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: myapp
spec:
  replicas: 3
  selector:
    matchLabels:
      app: myapp
  template:
    metadata:
      labels:
        app: myapp
    spec:
      containers:
        - name: myapp
          image: myapp:latest
          resources:
            limits:
              cpu: 500m
            requests:
              cpu: 200m
```

The following YAML defines the Horizontal Pod Autoscaler to ensure the Deployment can scale based on CPU utilization:

```
# hpa.yaml
apiVersion: autoscaling/v2beta2
kind: HorizontalPodAutoscaler
metadata:
  name: myapp-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: myapp
  minReplicas: 1
  maxReplicas: 10
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 50
```

Explanation

- The `scaleTargetRef` section specifies that the HPA targets the Deployment named "myapp".
- The autoscaler will maintain the replica count between 1 and 10 based on CPU usage.
- It continuously monitors the average CPU utilization and aims to maintain a 50% utilization threshold across all pods.

Managing the HPA

To create the HPA, run the following command:

```
kubectl create -f hpa.yaml
```

Upon successful creation, you will receive a confirmation message:

```
HorizontalPodAutoscaler Created
```

To view details of the created HPA, use the following command:

```
kubectl get hpa
```

The expected output might resemble:

NAME	REFERENCE	TARGETS	MINPODS	MAXPODS	REPLICAS	AGE
myapp-hpa	Deployment/myapp	50%/50%	1	10	3	10m

In the output:

- The "TARGETS" column shows both the current and the desired average CPU utilization.
- "MINPODS" and "MAXPODS" indicate the minimum and maximum replica limits.
- "REPLICAS" reflects the current number of running pods.
- "AGE" shows the duration for which the HPA has been active.



Caution

Ensure your metrics server is properly configured and running, as the HPA relies on accurate metrics to make scaling decisions.

To remove the HPA, execute the following command:

```
kubectl delete hpa myapp-hpa
```

Conclusion

This lesson demonstrated how to configure and monitor a Horizontal Pod Autoscaler for a Kubernetes Deployment. By dynamically adjusting the number of pods according to real-time resource usage, Kubernetes ensures that your application efficiently handles varying loads while optimizing resource use.



Watch Video

Watch video content

Previous

◀ Autoscaling

Next

Vertical Pod Autoscaler ▶



Beta



● Search docs...



Kubernetes and Cloud Native Associate - KCNA ▾